

ASSIGNMENT – 2

Regression Model Evaluation and Performance Analysis

AIM

To implement a regression model for predicting a continuous target variable, evaluate its performance using multiple evaluation metrics, visualize prediction results, and analyze overfitting, underfitting, and bias-variance. The dataset used is a real-world loan-amount prediction dataset (Kaggle: Predict Loan Amount Data) containing numerical and categorical features related to loan applications, with the target variable being the loan amount sanctioned. Linear Regression is used as the baseline model, and Ridge, Lasso, and Elastic Net regression are used as regularized models, tuned using GridSearchCV, to control model complexity and reduce overfitting. Libraries used: pandas, numpy, matplotlib, scikit-learn.

CODE

```
import time
import pandas as pd
import numpy as np
import matplotlib
matplotlib.use("Agg") # no pop-up windows, saves plots directly
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import LabelEncoder, StandardScaler

from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

TARGET = "Loan Sanction Amount (USD)"

# load the dataset
df = pd.read_csv("./train.csv")

# quick look at the data
print(df.head())
print(df.info())
print(df.describe())

# -----
# data preprocessing
# -----

# fill missing numeric values with median (median is safer from outliers)
num_cols = df.select_dtypes(include=["number"]).columns
num_cols = num_cols.drop([TARGET]) if TARGET in num_cols else num_cols
df[num_cols] = df[num_cols].fillna(df[num_cols].median())

# fill missing categorical values with mode (most common value)
cat_cols = df.select_dtypes(include=["object", "string"]).columns
for col in cat_cols:
    if not df[col].mode().empty:
        df[col] = df[col].fillna(df[col].mode()[0])
```

```
print("\nMissing values after imputation:")
print(df.isnull().sum())

# Customer ID and Name are just unique identifiers, they cant help prediction
df = df.drop(columns=["Customer ID", "Name"])

# drop rows with missing target
df = df.dropna(subset=[TARGET])

# encode categorical text columns to numbers
encoder = LabelEncoder()
for column in df.columns:
    if df[column].dtype == "object" or pd.api.types.is_string_dtype(df[column]):
        df[column] = encoder.fit_transform(df[column])

# separate features and target
X = df.drop(TARGET, axis=1)
y = df[TARGET]
feature_names = X.columns

# standardize so big numbers like income dont dominate smaller ones
scaler = StandardScaler()
X = scaler.fit_transform(X)

# -----
# exploratory data analysis (EDA)
# -----
print("\n===== EDA =====")
print("Summary statistics:")
print(df.describe())
print("\nDataset shape:", df.shape)

# target distribution
plt.figure(figsize=(6, 4))
plt.hist(y, bins=20)
plt.title("Loan Amount Distribution")
plt.xlabel("Loan Amount")
plt.ylabel("Frequency")
plt.show()
```

```
# feature distribution plots
grid_rows = int(np.ceil(len(feature_names) / 4))
plt.figure(figsize=(15, 10))
for i, column in enumerate(feature_names, 1):
    plt.subplot(grid_rows, 4, i)
    plt.hist(df[column], bins=20)
    plt.title(column)
    plt.xlabel(column)
    plt.ylabel("Frequency")
plt.tight_layout()
plt.show()

# feature vs target scatter plots
plt.figure(figsize=(15, 10))
for i, column in enumerate(feature_names, 1):
    plt.subplot(grid_rows, 4, i)
    plt.scatter(df[column], y, s=5, alpha=0.4)
    plt.title(column)
    plt.xlabel(column)
    plt.ylabel("Loan Amount")
plt.tight_layout()
plt.show()

# -----
# split into train / validation / test
# -----
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.30, random_state=42
)
X_validation, X_test, y_validation, y_test = train_test_split(
    X_temp, y_temp, test_size=0.50, random_state=42
)

print("\nTraining samples :", len(X_train))
print("Validation samples:", len(X_validation))
print("Testing samples   :", len(X_test))

# -----
# metrics helper
```

```
# -----
def calc_metrics(y_true, y_pred):
    mae = mean_absolute_error(y_true, y_pred)
    mse = mean_squared_error(y_true, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_true, y_pred)
    return mae, mse, rmse, r2

# -----
# 1. Linear Regression (baseline)
# -----
print("\n----- Linear Regression -----")
linear = LinearRegression()
start = time.time()
linear.fit(X_train, y_train)
linear_time = time.time() - start

val_pred_linear = linear.predict(X_validation)
test_pred_linear = linear.predict(X_test)

lin_val_mae, lin_val_mse, lin_val_rmse, lin_val_r2 = calc_metrics(y_validation, val_pred_linear)
lin_mae, lin_mse, lin_rmse, lin_r2 = calc_metrics(y_test, test_pred_linear)

print("Validation -> MAE:", lin_val_mae, "MSE:", lin_val_mse, "RMSE:", lin_val_rmse, "R2:", lin_val_r2)
print("Test      -> MAE:", lin_mae, "MSE:", lin_mse, "RMSE:", lin_rmse, "R2:", lin_r2)
print("Training Time :", linear_time)

# -----
# 2. Ridge Regression
# n_jobs=-1 uses all CPU cores so the grid search doesn't take forever
# -----
print("\n----- Ridge Regression -----")
ridge_model = Ridge()
parameters = {"alpha": [0.01, 0.1, 1, 10, 100]}
grid_ridge = GridSearchCV(ridge_model, parameters, cv=5, n_jobs=-1)
start = time.time()
grid_ridge.fit(X_train, y_train)
ridge_time = time.time() - start
```

```
print("Best Parameters:", grid_ridge.best_params_)
print("Best Cross Validation R2:", grid_ridge.best_score_)

val_pred_ridge = grid_ridge.predict(X_validation)
test_pred_ridge = grid_ridge.predict(X_test)

ridge_val_mae, ridge_val_mse, ridge_val_rmse, ridge_val_r2 = calc_metrics(y_validation, val_pred_ridge)
ridge_mae, ridge_mse, ridge_rmse, ridge_r2 = calc_metrics(y_test, test_pred_ridge)

print("Validation -> MAE:", ridge_val_mae, "MSE:", ridge_val_mse, "RMSE:", ridge_val_rmse, "R2:", ridge_val_r2)
print("Test -> MAE:", ridge_mae, "MSE:", ridge_mse, "RMSE:", ridge_rmse, "R2:", ridge_r2)

# -----
# 3. Lasso Regression
# -----
print("\n----- Lasso Regression -----")
lasso = Lasso(max_iter=5000)
lasso_params = {"alpha": [0.001, 0.01, 0.1, 1, 10]}
grid_lasso = GridSearchCV(lasso, lasso_params, cv=5, n_jobs=-1)
start = time.time()
grid_lasso.fit(X_train, y_train)
lasso_time = time.time() - start

print("Best Parameters :", grid_lasso.best_params_)
print("Best CV R2      :", grid_lasso.best_score_)

val_pred_lasso = grid_lasso.predict(X_validation)
test_pred_lasso = grid_lasso.predict(X_test)

lasso_val_mae, lasso_val_mse, lasso_val_rmse, lasso_val_r2 = calc_metrics(y_validation, val_pred_lasso)
lasso_mae, lasso_mse, lasso_rmse, lasso_r2 = calc_metrics(y_test, test_pred_lasso)

print("Validation -> MAE:", lasso_val_mae, "MSE:", lasso_val_mse, "RMSE:", lasso_val_rmse, "R2:", lasso_val_r2)
print("Test -> MAE:", lasso_mae, "MSE:", lasso_mse, "RMSE:", lasso_rmse, "R2:", lasso_r2)

# -----
# 4. Elastic Net Regression
```

```
# -----
print("\n----- Elastic Net Regression -----")
elastic = ElasticNet(max_iter=5000)
elastic_params = {
    "alpha": [0.01, 0.1, 1, 10],
    "l1_ratio": [0.2, 0.5, 0.8]
}
grid_elastic = GridSearchCV(elastic, elastic_params, cv=5, n_jobs=-1)
start = time.time()
grid_elastic.fit(X_train, y_train)
elastic_time = time.time() - start

print("Best Parameters :", grid_elastic.best_params_)
print("Best CV R2      :", grid_elastic.best_score_)

val_pred_elastic = grid_elastic.predict(X_validation)
test_pred_elastic = grid_elastic.predict(X_test)

elastic_val_mae, elastic_val_mse, elastic_val_rmse, elastic_val_r2 = calc_metrics(y_validation, val_pred_elastic)
elastic_mae, elastic_mse, elastic_rmse, elastic_r2 = calc_metrics(y_test, test_pred_elastic)

print("Validation -> MAE:", elastic_val_mae, "MSE:", elastic_val_mse, "RMSE:", elastic_val_rmse, "R2:", elastic_val_r2)
print("Test      -> MAE:", elastic_mae, "MSE:", elastic_mse, "RMSE:", elastic_rmse, "R2:", elastic_r2)

# -----
# Tables
# -----
table1 = pd.DataFrame({
    "Model": ["Ridge Regression", "Lasso Regression", "Elastic Net Regression"],
    "Search Method": ["Grid Search", "Grid Search", "Grid Search"],
    "Best Parameters": [grid_ridge.best_params_, grid_lasso.best_params_, grid_elastic.best_params_],
    "Best Cross Validation R2": [grid_ridge.best_score_, grid_lasso.best_score_, grid_elastic.best_score_]
})
print("\nTable 1: Hyperparameter Tuning Summary")
print(table1)

table2 = pd.DataFrame({
    "Model": ["Linear Regression", "Ridge Regression", "Lasso Regression", "Elastic Net"],
    "MAE": [lin_val_mae, ridge_val_mae, lasso_val_mae, elastic_val_mae],
```

```
"MSE": [lin_val_mse, ridge_val_mse, lasso_val_mse, elastic_val_mse],
"RMSE": [lin_val_rmse, ridge_val_rmse, lasso_val_rmse, elastic_val_rmse],
"R2 Score": [lin_val_r2, ridge_val_r2, lasso_val_r2, elastic_val_r2]
})
print("\nTable 2: Validation Performance Summary")
print(table2)

table3 = pd.DataFrame({
    "Model": ["Linear Regression", "Ridge Regression", "Lasso Regression", "Elastic Net"],
    "MAE": [lin_mae, ridge_mae, lasso_mae, elastic_mae],
    "MSE": [lin_mse, ridge_mse, lasso_mse, elastic_mse],
    "RMSE": [lin_rmse, ridge_rmse, lasso_rmse, elastic_rmse],
    "R2 Score": [lin_r2, ridge_r2, lasso_r2, elastic_r2],
    "Training Time (s)": [linear_time, ridge_time, lasso_time, elastic_time]
})
print("\nTable 3: Test Set Performance")
print(table3)

table4 = pd.DataFrame({
    "Feature": feature_names,
    "Linear": linear.coef_,
    "Ridge": grid_ridge.best_estimator_.coef_,
    "Lasso": grid_lasso.best_estimator_.coef_,
    "Elastic Net": grid_elastic.best_estimator_.coef_
})
print("\nTable 4: Coefficient Comparison")
print(table4)

# -----
# Visualizations: Predicted vs Actual and Residuals
# -----
model_preds = {
    "Linear Regression": test_pred_linear,
    "Ridge Regression": test_pred_ridge,
    "Lasso Regression": test_pred_lasso,
    "Elastic Net": test_pred_elastic
}

for name, preds in model_preds.items():
```

```
# predicted vs actual
plt.figure(figsize=(6, 6))
plt.scatter(y_test, preds, s=5, alpha=0.4)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], "r--", linewidth=1)
plt.xlabel("Actual Values")
plt.ylabel("Predicted Values")
plt.title(f"Actual vs Predicted - {name}")
plt.show()

# residual plot
residual = y_test - preds
plt.figure(figsize=(6, 4))
plt.scatter(preds, residual, s=5, alpha=0.4)
plt.axhline(y=0, color="r", linestyle="--")
plt.xlabel("Predicted Values")
plt.ylabel("Residual")
plt.title(f"Residual Plot - {name}")
plt.show()

# -----
# Training vs Validation Error across Ridge alphas
# -----
train_error = []
validation_error = []
alphas = [0.01, 0.1, 1, 10, 100]

for alpha in alphas:
    model = Ridge(alpha=alpha)
    model.fit(X_train, y_train)
    train_pred = model.predict(X_train)
    val_pred = model.predict(X_validation)
    train_error.append(mean_squared_error(y_train, train_pred))
    validation_error.append(mean_squared_error(y_validation, val_pred))

plt.figure(figsize=(6, 4))
plt.plot(alphas, train_error, marker="o", label="Training Error")
plt.plot(alphas, validation_error, marker="o", label="Validation Error")
plt.xscale("log")
plt.xlabel("Alpha")
```

```
plt.ylabel("MSE")
plt.title("Training vs Validation Error (Ridge)")
plt.legend()
plt.show()

# -----
# Coefficient comparison bar plot
# -----
models_list = ["Linear", "Ridge", "Lasso", "ElasticNet"]
coefficients = [
    np.mean(np.abs(linear.coef_)),
    np.mean(np.abs(grid_ridge.best_estimator_.coef_)),
    np.mean(np.abs(grid_lasso.best_estimator_.coef_)),
    np.mean(np.abs(grid_elastic.best_estimator_.coef_))
]

plt.figure(figsize=(6, 4))
plt.bar(models_list, coefficients)
plt.title("Coefficient Comparison (Average Magnitude)")
plt.ylabel("Average Coefficient Magnitude")
plt.show()

# -----
# Table 5: Overfitting / Underfitting check
# -----
train_pred_linear = linear.predict(X_train)
train_pred_ridge = grid_ridge.predict(X_train)
train_pred_lasso = grid_lasso.predict(X_train)
train_pred_elastic = grid_elastic.predict(X_train)

train_mse_linear = mean_squared_error(y_train, train_pred_linear)
train_mse_ridge = mean_squared_error(y_train, train_pred_ridge)
train_mse_lasso = mean_squared_error(y_train, train_pred_lasso)
train_mse_elastic = mean_squared_error(y_train, train_pred_elastic)

table5 = pd.DataFrame({
    "Model": ["Linear Regression", "Ridge Regression", "Lasso Regression", "Elastic Net"],
    "Training MSE": [train_mse_linear, train_mse_ridge, train_mse_lasso, train_mse_elastic],
```

```
"Validation MSE": [lin_val_mse, ridge_val_mse, lasso_val_mse, elastic_val_mse],
"Gap (Val - Train)": [
    lin_val_mse - train_mse_linear,
    ridge_val_mse - train_mse_ridge,
    lasso_val_mse - train_mse_lasso,
    elastic_val_mse - train_mse_elastic
]
})
print("\nTable 5: Training vs Validation MSE")
print(table5)

print("\nAll tables and plots generated successfully.")
```

OUTPUT

	Customer ID	Name	... Property Price	Loan Sanction Amount (USD)
0	C-36995	Frederica Shealy	... 119933.46	54607.18
1	C-33999	America Calderone	... 54791.00	37469.98
2	C-3770	Rosetta Verne	... 72440.58	36474.43
3	C-26480	Zoe Chitty	... 121441.51	56040.54
4	C-23459	Afton Venema	... 208567.91	74008.28

[5 rows x 24 columns]

<class 'pandas.DataFrame'>

RangeIndex: 30000 entries, 0 to 29999

Data columns (total 24 columns):

#	Column	Non-Null Count	Dtype
0	Customer ID	30000 non-null	str
1	Name	30000 non-null	str
2	Gender	29947 non-null	str
3	Age	30000 non-null	int64
4	Income (USD)	25424 non-null	float64
5	Income Stability	28317 non-null	str
6	Profession	30000 non-null	str
7	Type of Employment	22730 non-null	str
8	Location	30000 non-null	str
9	Loan Amount Request (USD)	30000 non-null	float64
10	Current Loan Expenses (USD)	29828 non-null	float64
11	Expense Type 1	30000 non-null	str
12	Expense Type 2	30000 non-null	str
13	Dependents	27507 non-null	float64
14	Credit Score	28297 non-null	float64
15	No. of Defaults	30000 non-null	int64
16	Has Active Credit Card	28434 non-null	str
17	Property ID	30000 non-null	int64
18	Property Age	25150 non-null	float64
19	Property Type	30000 non-null	int64
20	Property Location	29644 non-null	str
21	Co-Applicant	30000 non-null	int64

```
22 Property Price          30000 non-null float64
23 Loan Sanction Amount (USD) 29660 non-null float64
dtypes: float64(8), int64(5), str(11)
memory usage: 5.5 MB
None
```

	Age	Income (USD)	...	Property Price	Loan Sanction Amount (USD)
count	30000.000000	2.542400e+04	...	3.000000e+04	29660.000000
mean	40.092300	2.630574e+03	...	1.317597e+05	47649.342208
std	16.045129	1.126272e+04	...	9.354955e+04	48221.146686
min	18.000000	3.777000e+02	...	-9.990000e+02	-999.000000
25%	25.000000	1.650457e+03	...	6.057216e+04	0.000000
50%	40.000000	2.222435e+03	...	1.099936e+05	35209.395000
75%	55.000000	3.090593e+03	...	1.788807e+05	74261.250000
max	65.000000	1.777460e+06	...	1.077967e+06	481907.320000

[8 rows x 13 columns]

Missing values after imputation:

Customer ID	0
Name	0
Gender	0
Age	0
Income (USD)	0
Income Stability	0
Profession	0
Type of Employment	0
Location	0
Loan Amount Request (USD)	0
Current Loan Expenses (USD)	0
Expense Type 1	0
Expense Type 2	0
Dependents	0
Credit Score	0
No. of Defaults	0
Has Active Credit Card	0

```
Property ID          0
Property Age         0
Property Type        0
Property Location    0
Co-Applicant         0
Property Price       0
Loan Sanction Amount (USD)  340
dtype: int64
```

===== EDA =====

Summary statistics:

	Gender	Age	...	Property Price	Loan Sanction Amount (USD)
count	29660.000000	29660.000000	...	2.966000e+04	29660.000000
mean	0.503776	40.099225	...	1.317335e+05	47649.342208
std	0.499994	16.050024	...	9.342129e+04	48221.146686
min	0.000000	18.000000	...	-9.990000e+02	-999.000000
25%	0.000000	25.000000	...	6.065874e+04	0.000000
50%	1.000000	40.000000	...	1.099936e+05	35209.395000
75%	1.000000	55.000000	...	1.788019e+05	74261.250000
max	1.000000	65.000000	...	1.077967e+06	481907.320000

[8 rows x 22 columns]

Dataset shape: (29660, 22)

[captured plot 1 -> plot_01.png]

[captured plot 2 -> plot_02.png]

[captured plot 3 -> plot_03.png]

Training samples : 20762

Validation samples: 4449

Testing samples : 4449

----- Linear Regression -----

Validation -> MAE: 21532.77686106947 MSE: 1006106474.0251105 RMSE: 31719.181484160505 R2: 0.5560315163206522

Test -> MAE: 21655.570817934487 MSE: 979948821.8642541 RMSE: 31304.134261535713 R2: 0.5710039098377462

Training Time : 0.009625434875488281

----- Ridge Regression -----

Best Parameters: {'alpha': 100}

Best Cross Validation R2: 0.5746289509603069

Validation -> MAE: 21556.606595865695 MSE: 1006358175.1601073 RMSE: 31723.148884688406 R2: 0.5559204471901676

Test -> MAE: 21666.006242453655 MSE: 979514292.3781044 RMSE: 31297.193043116575 R2: 0.5711941355377615

----- Lasso Regression -----

Best Parameters : {'alpha': 10}

Best CV R2 : 0.5689437843653605

Validation -> MAE: 21524.674709667106 MSE: 1006048774.0252962 RMSE: 31718.2719268452 R2: 0.556056977822081

Test -> MAE: 21653.02986942589 MSE: 979991617.1548274 RMSE: 31304.817794627514 R2: 0.570985175173319

----- Elastic Net Regression -----

Best Parameters : {'alpha': 0.1, 'l1_ratio': 0.5}

Best CV R2 : 0.5850602654047347

Validation -> MAE: 21790.704794838057 MSE: 1014925260.9750767 RMSE: 31857.89165929027 R2: 0.5521400161950204

Test -> MAE: 21804.313415670695 MSE: 984312132.0522676 RMSE: 31373.749091434187 R2: 0.5690937661965079

Table 1: Hyperparameter Tuning Summary

	Model	... Best Cross Validation R2
0	Ridge Regression	0.574629
1	Lasso Regression	0.568944
2	Elastic Net Regression	0.585060

[3 rows x 4 columns]

Table 2: Validation Performance Summary

	Model	MAE	MSE	RMSE	R2 Score
0	Linear Regression	21532.776861	1.006106e+09	31719.181484	0.556032
1	Ridge Regression	21556.606596	1.006358e+09	31723.148885	0.555920
2	Lasso Regression	21524.674710	1.006049e+09	31718.271927	0.556057
3	Elastic Net	21790.704795	1.014925e+09	31857.891659	0.552140

Table 3: Test Set Performance

	Model	MAE	...	R2 Score	Training Time (s)
0	Linear Regression	21655.570818	...	0.571004	0.009625
1	Ridge Regression	21666.006242	...	0.571194	12.876722
2	Lasso Regression	21653.029869	...	0.570985	7.397294
3	Elastic Net	21804.313416	...	0.569094	3.185498

[4 rows x 6 columns]

Table 4: Coefficient Comparison

	Feature	Linear	...	Lasso	Elastic Net
0	Gender	-67.618433	...	-55.368878	-32.736948
1	Age	-540.560156	...	-523.974906	-516.958832
2	Income (USD)	42849.833252	...	17.082970	109.748397
3	Income Stability	847.438815	...	840.185341	675.062715
4	Profession	-279.031709	...	-269.957871	-320.492703
5	Type of Employment	325.889007	...	314.753304	232.478573
6	Location	-166.121181	...	-132.504705	-40.169546
7	Loan Amount Request (USD)	34930.939839	...	34729.133652	24915.287185
8	Current Loan Expenses (USD)	-1048.204094	...	-996.658333	509.271390
9	Expense Type 1	4.457602	...	9.449152	42.014300
10	Expense Type 2	291.835578	...	287.934407	255.301315
11	Dependents	24.539120	...	18.010895	41.828002
12	Credit Score	11996.574835	...	11982.443737	11562.967865
13	No. of Defaults	-298.817217	...	-292.379355	-311.223783
14	Has Active Credit Card	131.682313	...	124.970619	89.248697
15	Property ID	-48.509964	...	-41.340671	8.985655
16	Property Age	-42825.712372	...	0.000000	-91.658415
17	Property Type	-157.203078	...	-147.998253	-169.013327
18	Property Location	213.957137	...	200.947205	231.508794
19	Co-Applicant	-497.569623	...	-487.822719	-517.185616
20	Property Price	-963.754949	...	-771.348593	7259.407609

[21 rows x 5 columns]

[captured plot 4 -> plot_04.png]

```
[captured plot 5 -> plot_05.png]
[captured plot 6 -> plot_06.png]
[captured plot 7 -> plot_07.png]
[captured plot 8 -> plot_08.png]
[captured plot 9 -> plot_09.png]
[captured plot 10 -> plot_10.png]
[captured plot 11 -> plot_11.png]
[captured plot 12 -> plot_12.png]
[captured plot 13 -> plot_13.png]
```

Table 5: Training vs Validation MSE

	Model	Training MSE	Validation MSE	Gap (Val - Train)
0	Linear Regression	9.520808e+08	1.006106e+09	5.402563e+07
1	Ridge Regression	9.527562e+08	1.006358e+09	5.360197e+07
2	Lasso Regression	9.525070e+08	1.006049e+09	5.354175e+07
3	Elastic Net	9.622232e+08	1.014925e+09	5.270210e+07

All tables and plots generated successfully.

PLOTS

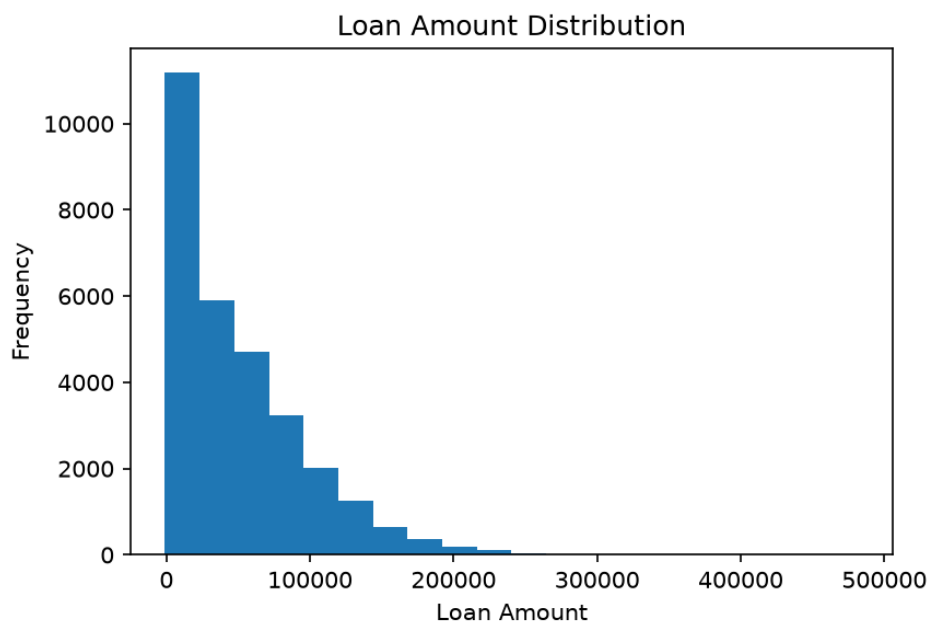


Fig 1 - Loan Amount Distribution (target variable)

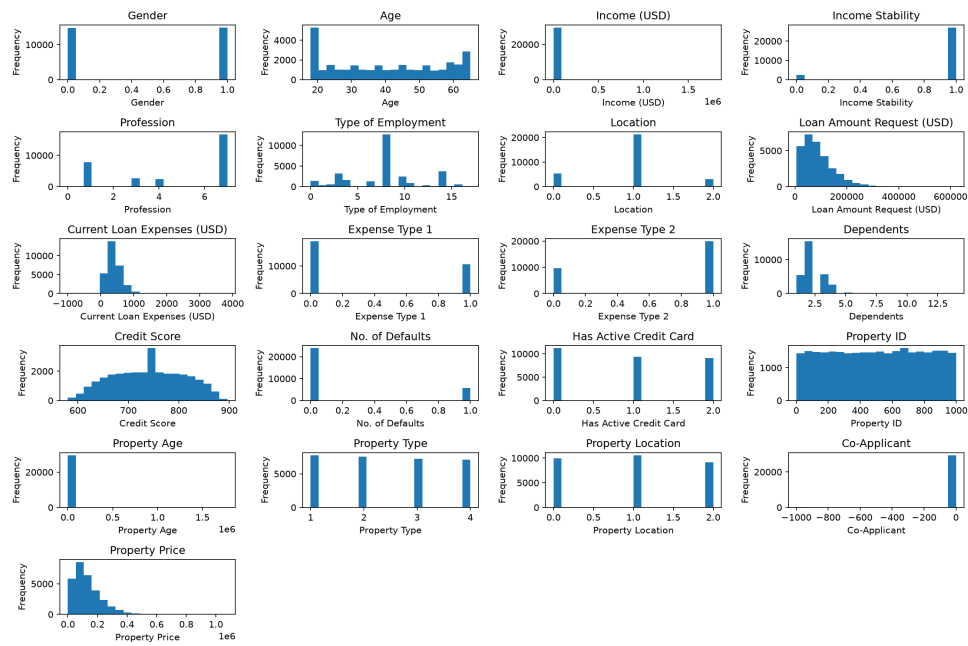


Fig 2 – Feature distribution histograms (all 21 features)

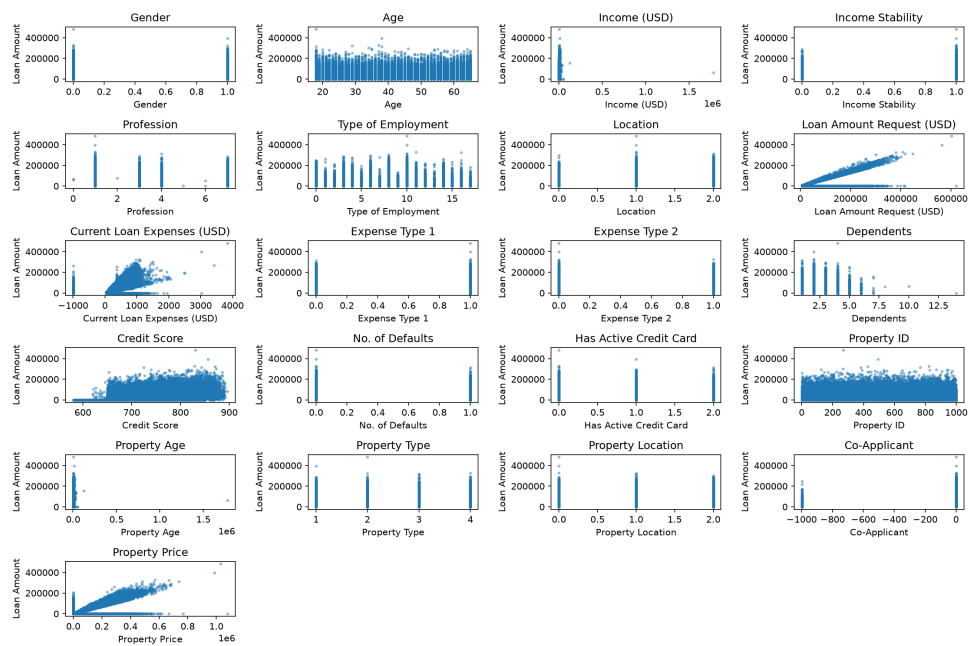


Fig 3 – Feature vs Target scatter plots (all 21 features)

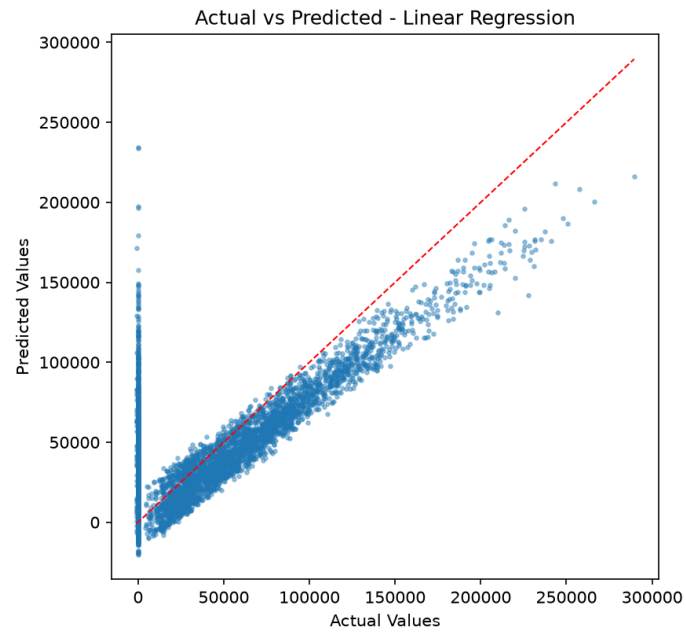


Fig 4 - Actual vs Predicted - Linear Regression

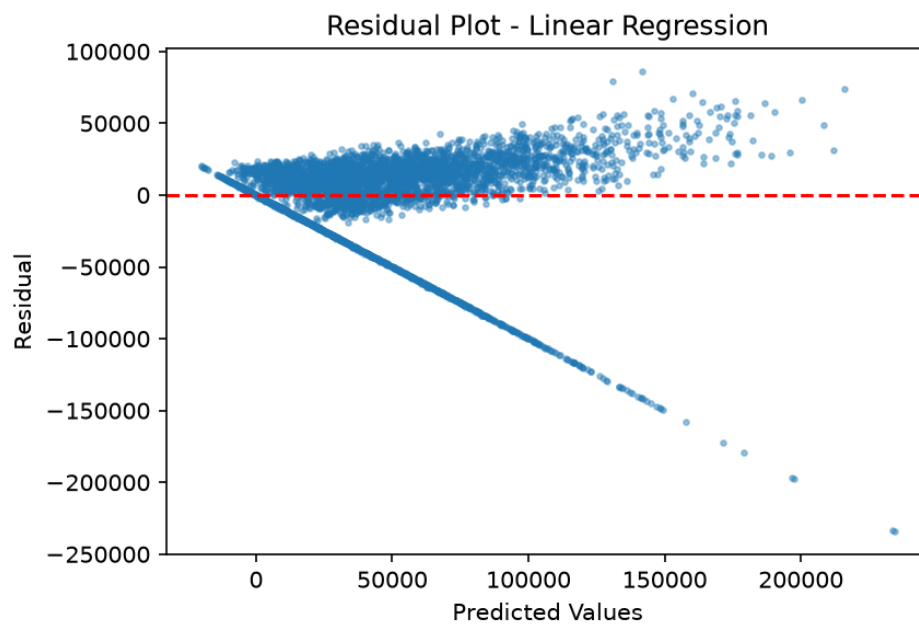


Fig 5 - Residual Plot - Linear Regression

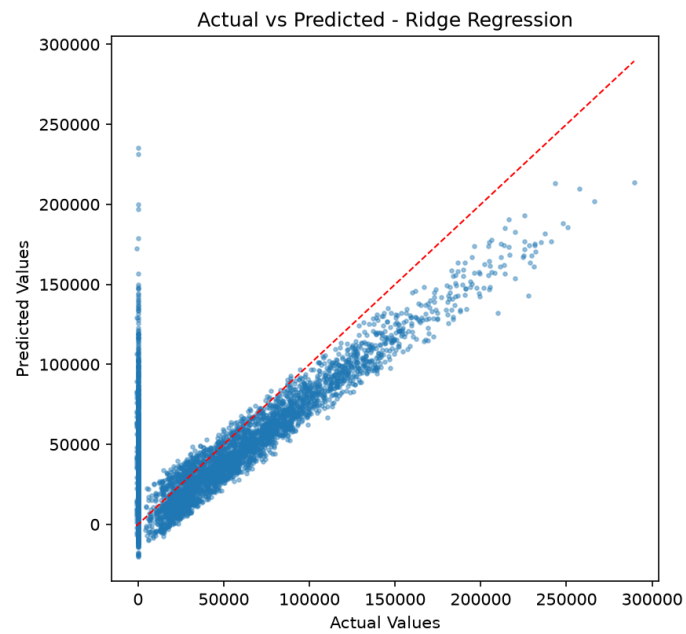


Fig 6 - Actual vs Predicted - Ridge Regression

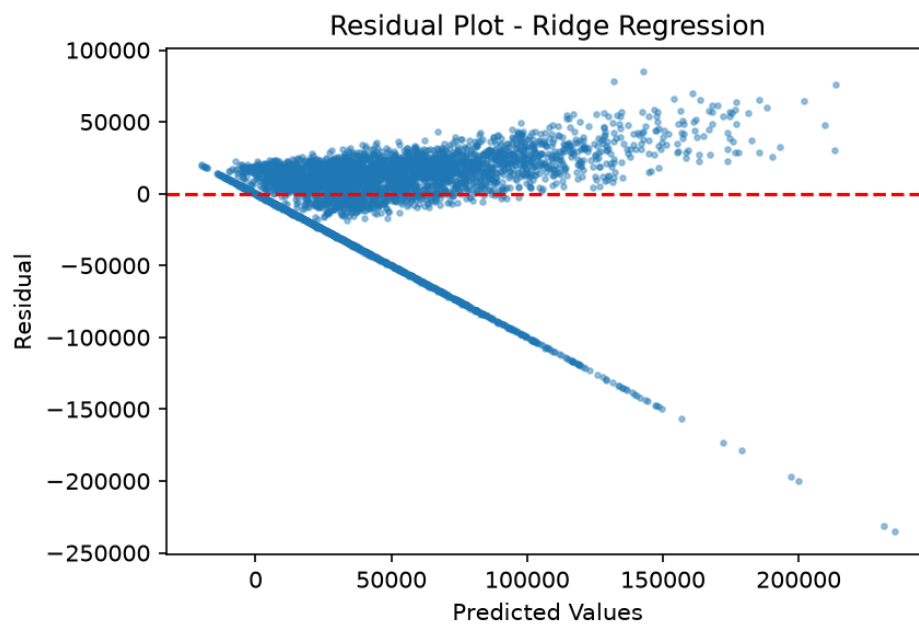


Fig 7 - Residual Plot - Ridge Regression

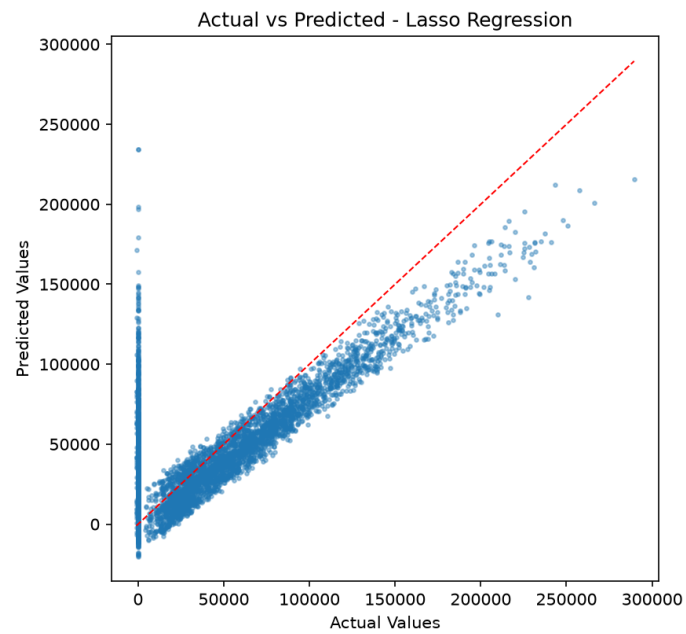


Fig 8 - Actual vs Predicted - Lasso Regression

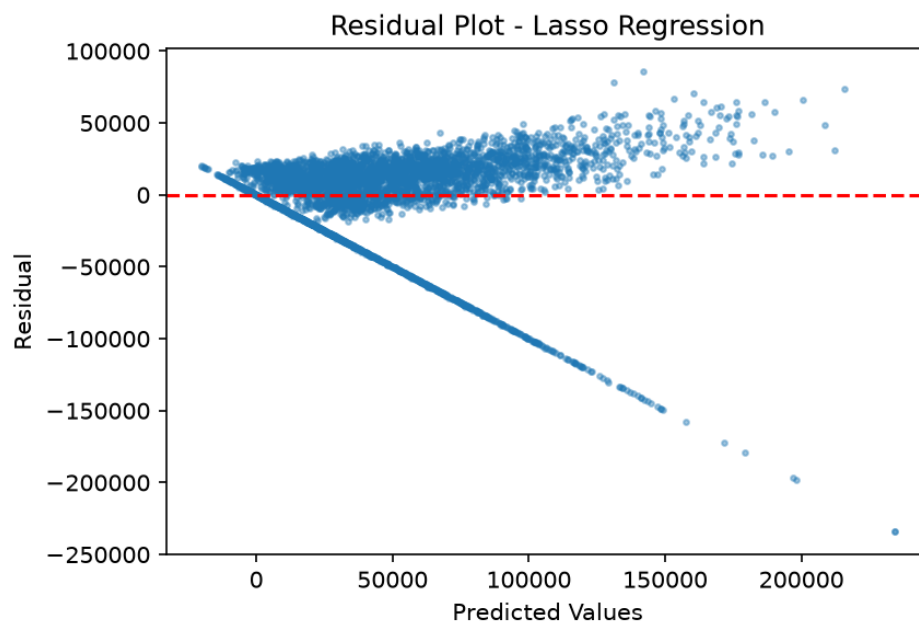


Fig 9 - Residual Plot - Lasso Regression

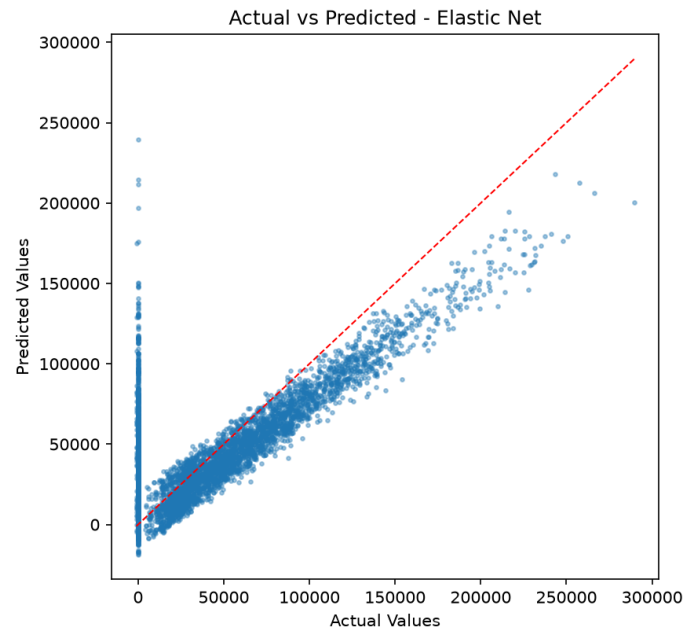


Fig 10 - Actual vs Predicted - Elastic Net

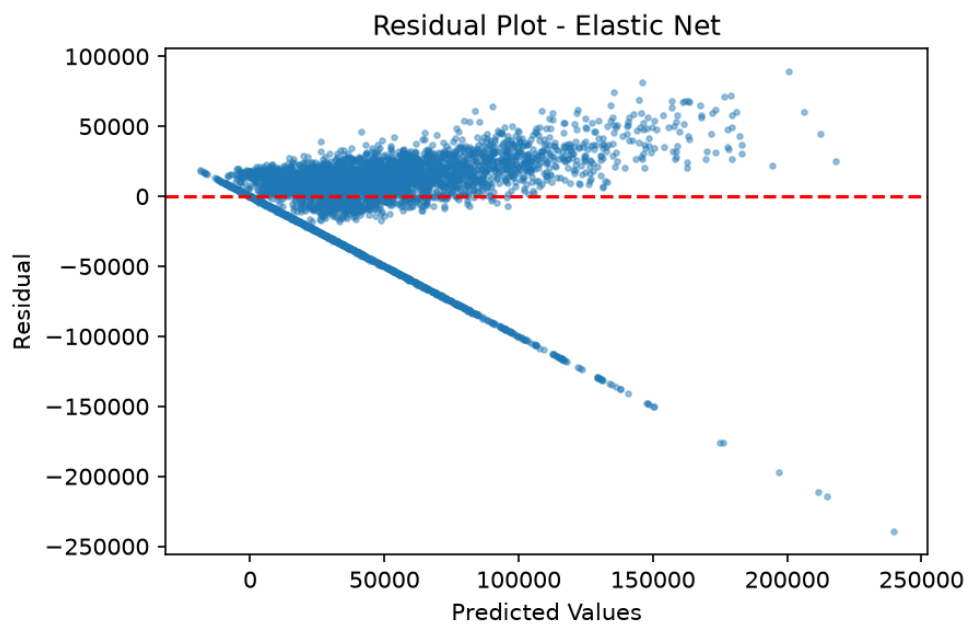
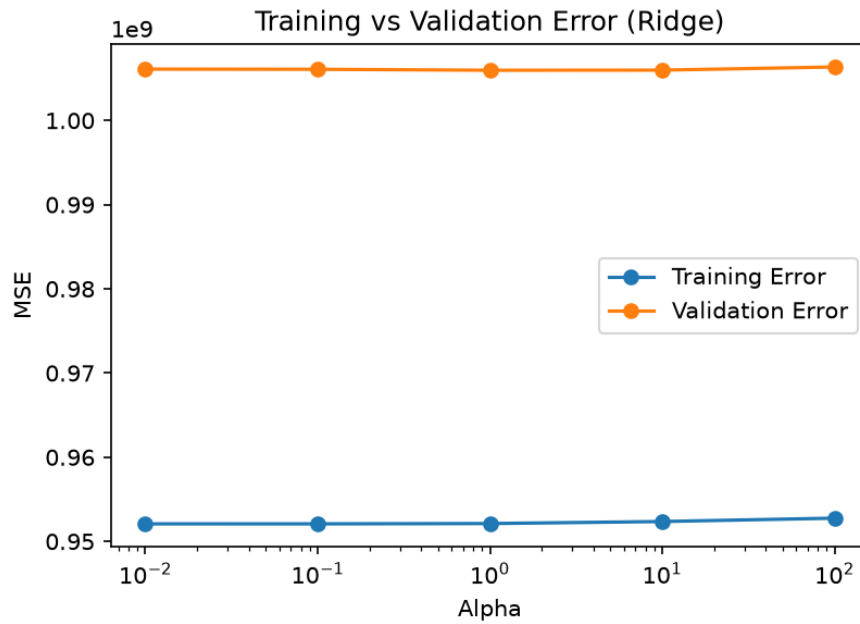
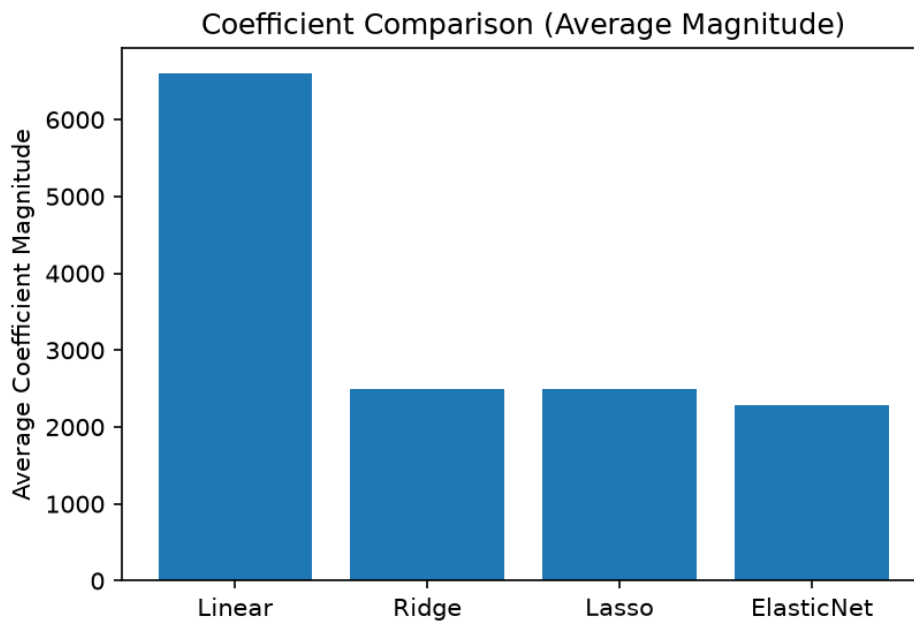


Fig 11 - Residual Plot - Elastic Net

*Fig 12 - Training vs Validation Error (Ridge)**Fig 13 - Coefficient Comparison (average magnitude) across models*

TASKS PERFORMED

Task	Description	Result / Output
Loaded and inspected the dataset	Read train.csv with pandas and checked info/describe	30000 rows x 24 columns; 8 float, 5 int, 11 string columns
Handled missing values	Filled numeric NaNs with median; filled categorical NaNs with mode; dropped rows with missing target	0 missing values except 340 target rows which were dropped; final shape (29660, 22)

Task	Description	Result / Output
Dropped irrelevant columns	Removed Customer ID and Name because they are just unique identifiers	21 features used for modeling
Encoded categorical variables	LabelEncoder on remaining text columns	All columns converted to numeric
Standardized numerical features	StandardScaler on all feature columns	Features scaled to mean 0 and std 1
Split the dataset	70/15/15 split with random_state=42	20762 training, 4449 validation, 4449 testing samples
Exploratory Data Analysis	Plotted target distribution, feature histograms, and feature-vs-target scatter plots	Figs 1, 2, 3
Trained Linear Regression	Baseline model on standardized data	Test R2 = 0.5710, training time 0.010 s
Tuned Ridge Regression	GridSearchCV over alpha = [0.01, 0.1, 1, 10, 100] with cv=5	Best alpha = 100, Test R2 = 0.5712
Tuned Lasso Regression	GridSearchCV over alpha = [0.001, 0.01, 0.1, 1, 10] with cv=5	Best alpha = 10, Test R2 = 0.5710
Tuned Elastic Net Regression	GridSearchCV over alpha x l1_ratio grid with cv=5	Best alpha = 0.1, l1_ratio = 0.5, Test R2 = 0.5691
Compared coefficients	Built a per-feature table for Linear, Ridge, Lasso, and Elastic Net	Table 4 (21 rows x 5 columns)
Visualized predictions and residuals	Actual-vs-Predicted and Residual plots for all 4 models	Figs 4-11
Analyzed overfitting/underfitting	Compared training vs validation MSE across all models and Ridge alpha values	Table 5; validation MSE slightly higher than training MSE

LEARNING OUTCOMES

- I was able to preprocess a real-world dataset by imputing missing values, encoding categorical features, and standardizing numerical features before feeding them to a model.

- I understood how to split a dataset into training, validation, and test sets for proper model evaluation.
- I implemented and trained a baseline Linear Regression model for a continuous target variable.
- I applied Ridge, Lasso, and Elastic Net regularization to control model complexity and reduce overfitting.
- I used GridSearchCV with 5-fold cross-validation to tune hyperparameters such as alpha and l1_ratio.
- I evaluated regression models using MAE, MSE, RMSE, and R2 score on both validation and test sets.
- I visualized predicted-vs-actual values and residuals to assess how well each model fit the data.
- I compared training and validation error to discuss overfitting and underfitting behaviour.